

每日资讯摘要订阅系统

深度学习与自然语言处理—课程设计（题目 4）

姓名：奚俊戈 学号：23089052

组员：顾昊昱 学号：23089025

2026 年 6 月

1 选题背景与目标

1.1 问题背景

在信息爆炸的时代，用户每天面临海量新闻资讯，难以高效获取真正有价值的内容。传统的 RSS 阅读器仅做信息聚合，缺乏智能摘要和自动分类能力。用户不得不花费大量时间逐篇浏览，信息过载已成为现代人的普遍痛点。

自动文本摘要技术正是解决这一问题的关键。当前主流方案分为两条技术路线：一是**抽取式摘要**——从原文中直接选取关键句拼接成摘要，以 TextRank 算法为代表；二是**生成式摘要**——由模型理解原文后重新组织语言生成摘要，以 Transformer 架构的大语言模型为代表。两种方案各有优劣：抽取式忠实于原文但缺乏概括能力；生成式灵活自然但可能引入事实偏差。

1.2 项目目标

本项目构建一个端到端的每日资讯摘要订阅系统，实现以下核心功能：

- 多源抓取**：自动从 Hacker News API 和 36 氪 RSS 获取每日资讯，利用 trafilatura 提取正文
- 双方案摘要**：实现 TextRank 抽取式（自实现）和 LLM 生成式（Qwen2.5:7B）两种摘要方案
- 自动分类**：利用 LLM 将文章归入四类——AI/科技、创业/商业、编程/技术、其他
- 多长度档位**：支持一句话摘要、约 100 字短摘要、约 200 字详细摘要
- Web 展示**：基于 Streamlit 构建可交互界面，支持按日期回顾、按分类筛选、摘要方案对比
- 量化评估**：使用 ROUGE-L 指标和人工评分对两种摘要方案进行对比

1.3 涉及课程知识

本项目综合运用了本学期 NLP 课程中多个模块的知识，具体对应关系如表 1 所示。

表 1: 课程知识在本项目中的应用

课程模块	对应实现
词向量 / TF-IDF	TextRank 句子向量化，计算句子间相似度
Seq2Seq / Attention	Qwen2.5 生成式摘要的底层架构原理
Transformer	大语言模型的核心架构，理解 LLM 工作机制
Prompt Engineering	设计 System Prompt 控制分类与摘要输出格式
结构化输出	LLM 按 JSON 格式输出 {category, summaries}，便于下游解析
工程实践	模块化架构、SQLite 持久化、Streamlit 前端

2 相关技术介绍

2.1 TextRank 抽取式摘要

TextRank 算法由 Mihalcea 和 Tarau 于 2004 年提出 [1]，其核心思想源自 Google 的 PageRank 算法 [2]：将文本中的句子视为图的节点，句子之间的语义相似度作为边权重，通过迭代计算每个句子的重要性得分，最后选取得分最高的若干句子按原文顺序排列形成摘要。

算法流程分为五个步骤：

- 步骤 1: **分句**：按中英文标点（句号、问号、感叹号等）将原文切分为句子列表
- 步骤 2: **分词**：使用 jieba 对每个句子进行中文分词，同时过滤停用词（的、了、在、是、the、a 等）
- 步骤 3: **向量化**：计算每个句子的 TF-IDF 向量。TF（词频）衡量词在句子中的重要性，IDF（逆文档频率）降低高频低信息量词汇的权重
- 步骤 4: **相似度矩阵**：计算句子两两之间的余弦相似度，构建 $N \times N$ 的邻接矩阵（对角线置零，消除自相似）
- 步骤 5: **PageRank 迭代**：以阻尼系数 $d = 0.85$ 进行迭代，直至分数收敛（容差 10^{-6} ）。选取 Top-K 个句子，按原文顺序排列输出

TextRank 的优势在于无监督、无需训练数据、计算速度快（纯 CPU 的 < 1 秒可处理单篇文章）。但其局限也很明显：摘要是原文片段的拼接，无法生成新的概括性语句；对文章结构的依赖性强，可能选到非核心句（如对话片段）。

2.2 LLM 生成式摘要

本项目使用阿里通义千问系列模型 Qwen2.5:7B[5]，通过 Ollama 在本地部署运行。Qwen2.5 基于 Transformer Decoder 架构，具有 3584 维嵌入、32K 上下文窗口，支持中英双语。模型以 Q4_K_M 量化格式运行，占用约 4.7GB 显存，在 NVIDIA RTX 4070 (8GB 显存) 上推理流畅。

生成式摘要的工作原理是：通过精心设计的 System Prompt 引导模型阅读原文，然后自回归式地生成摘要文本。与抽取式不同，LLM 可以理解语义、归纳要点、重新组织语言，生成更接近人工撰写的摘要。

为了让一次 LLM 调用同时完成多项任务，我们设计了结构化 JSON 输出的 Prompt 策略：System Prompt 要求模型输出包含 `category` (分类)、`one_sentence` (一句话摘要)、`short` (约 100 字摘要) 和 `long` (约 200 字摘要) 的 JSON 对象。输入正文截断至前 3000 字符——既保留足够上下文供模型理解，又避免超出上下文窗口。temperature 设为 0.3，在创造性和确定性之间取得平衡。

2.3 ROUGE 评估指标

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) [3] 是自动文本摘要领域最广泛使用的评估指标。本项目使用 ROUGE-L 变体——基于最长公共子序列 (Longest Common Subsequence, LCS) 计算生成摘要与人工参考摘要之间的匹配度：

$$R_{LCS} = \frac{LCS(X,Y)}{|Y|}, \quad P_{LCS} = \frac{LCS(X,Y)}{|X|}, \quad F_{LCS} = \frac{2 \cdot R_{LCS} \cdot P_{LCS}}{R_{LCS} + P_{LCS}} \quad (1)$$

其中 X 为生成摘要， Y 为参考摘要。需要注意的是，中文文本需要先经过 jieba 分词后再计算 ROUGE——因为标准 ROUGE 实现基于英文的空格分词逻辑，中文词之间没有天然分隔。

2.4 正文提取

正文提取采用 trafilatura[6]——一个现代的 Python Web 内容提取库。它能够自动识别网页中的正文区域，过滤导航栏、广告、推荐链接等噪声。选择 trafilatura 而非更流行的 newspaper3k，是因为后者及其依赖 (feedparser) 在 Python 3.13 下存在 sgmlib 兼容问题。

RSS 解析直接使用 lxml.etree 解析 XML，支持 RSS 2.0 和 Atom 两种格式，避免了 feedparser 在 Python 3.13 下的相同兼容问题。

3 系统设计

3.1 系统架构

系统采用分层架构设计，自底向上分为数据层、业务逻辑层和表现层，如图 1 所示。

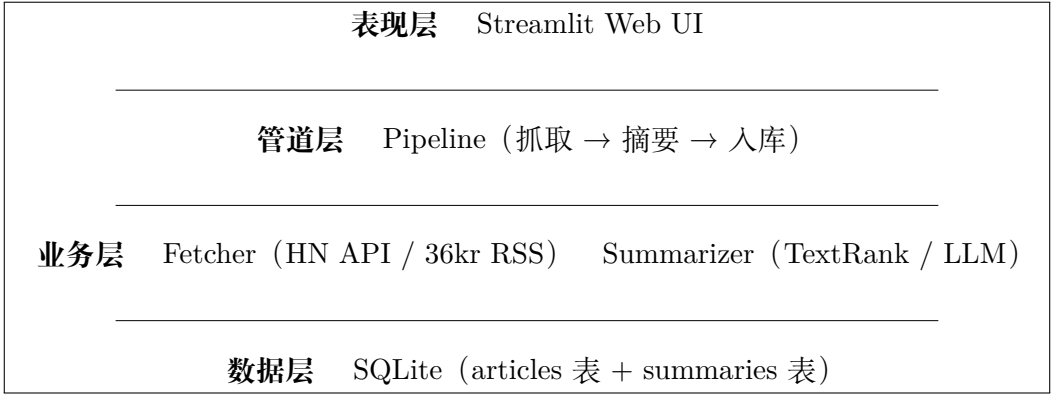


图 1: 系统架构分层图

3.2 模块划分

系统共包含 6 个模块，文件结构和职责如表 2 所示。

表 2: 模块划分与职责

模块	文件	职责
config	config.py	全局配置（模型名、信息源、分类标签、DB 路径）
fetcher	hackernews.py / rss_36kr.py	信息源抓取 + 正文提取
summarizer	textrank.py / llm_summarizer.py	TextRank 自实现 + LLM 摘要生成
storage	db.py	SQLite 持久化，CRUD 操作
pipeline	pipeline.py	每日流水线：（1）抓取 -> 入库，（2）摘要 -> 入库
ui	main.py	Streamlit 4 页 Web 界面

3.3 数据库设计

数据库使用 SQLite，包含两张核心表，通过 `article_id` 外键关联。

articles 表存储原始文章：`id`（主键）、`source`（来源名称）、`title`、`url`、`content`（正文）、`raw_summary`（RSS 简介）、`fetch_date`（抓取日期）、`fetched_at`（抓取时间戳）。`UNIQUE(source, url)` 约束防止同一文章重复入库。

summaries 表存储摘要结果：`id`（主键）、`article_id`（外键，`UNIQUE`）、`category`（分类标签）、`one_sentence`、`short`、`long`（LLM 三种长度摘要）、`textrank`（TextRank 抽取式摘要）、`created_at`。通过 `LEFT JOIN` 可在一次查询中同时获取文章和摘要信息。

3.4 数据流

完整的数据流如下：系统从 Hacker News API 和 36 氪 RSS 两个信息源抓取文章 → trafilatura 提取正文 → 存入 articles 表 → Pipeline 触发摘要生成 → TextRank 抽取关键句 + LLM 生成三种长度摘要并分类 → 存入 summaries 表 → Streamlit 前端按日期/分类读取展示。

4 关键实现细节

4.1 TextRank 自实现

与直接调用 sumy 等现成库不同，本项目从零实现了 TextRank 算法，以体现对算法原理的深入理解。核心实现在 `textrank.py` 中，约 80 行代码，涵盖分句、分词、TF-IDF 向量化、余弦相似度矩阵构建和 PageRank 迭代五个步骤。

TF-IDF 向量化是关键设计选择。如果仅使用词频 (TF) 向量，高频但低信息量的停顿词（如“的”“了”）会主导相似度计算；加入 IDF 权重后，这些词被大幅降权，真正承载语义的实词（如“Transformer”“摘要”“模型”）在相似度计算中占据主导。

PageRank 迭代设置阻尼系数 $d = 0.85$ （经典值），收敛容差 10^{-6} ，最大迭代次数 100。在 20 句以内的短文本上，通常 10-20 次迭代即可收敛。

4.2 LLM Prompt 设计

一次 LLM 调用同时完成**分类**和**三种长度摘要**，避免了多次调用带来的延迟和 token 浪费。System Prompt 明确指定输出格式为 JSON，并约束类别范围（AI/科技、创业/商业、编程/技术、其他）。在解析阶段，首先用正则提取 JSON 代码块（兼容 LLM 偶尔输出的 ````json ... ```` 包裹），再通过规范化步骤将非标准分类修正为“其他”。

4.3 数据抓取策略

HN API 的抓取流程：获取 `/topstories.json` → 取前 30 个 ID → 逐个查询 `/item/{id}.json` → 提取 title 和 url → trafilatura 抓取正文。每篇文章独立抓取，单篇失败不影响其他。

36kr RSS 的抓取流程：GET 请求获取 RSS XML → lxml.etree 解析 → 提取 `<item>` 元素中的 title、link、description → trafilatura 抓取正文。设计了降级策略：如果正文提取失败但 RSS 简介 100 字，则用简介替代正文，保证覆盖率。

正文质量通过长度阈值过滤：英文文章 100 字符，中文文章 50 字符——因为中文信息密度更高，同样的信息量需要更少的字符数。

5 实验与评估

为全面评估两种摘要方案的效果，我们设计了三种评估方式：ROUGE-L 自动评估（20 篇文章）、人工多维评分（5 篇文章）、分类准确率验证（20 篇文章）。人工参考摘要由项目组成员独立撰写，确保评估的客观性。

5.1 ROUGE-L 评估

在 20 篇文章上，以人工撰写的参考摘要为 ground truth，计算 TextRank 和 LLM 生成式摘要的 ROUGE-L F1 分数。

表 3: ROUGE-L 评估结果（20 篇文章）

方案	文章数	ROUGE-L F1
TextRank 抽取式	20	0.1322
LLM 生成式（100 字）	20	0.3571
LLM 生成式（200 字）	20	0.2562

结果分析：

- (1) **LLM 显著优于 TextRank**：LLM 100 字摘要的 ROUGE-L F1（0.3571）是 TextRank（0.1322）的 2.7 倍。这是因为抽取式摘要直接使用原文原句，而人工参考摘要重新组织了语言——即使信息覆盖到位，n-gram 层面的重叠度也天然偏低。
- (2) **200 字摘要 ROUGE-L 反而更低**：LLM 200 字摘要的 ROUGE-L（0.2562）低于 100 字（0.3571）。这是 ROUGE 指标本身的局限性——更长文本与参考摘要的精确匹配概率更低。实际上，200 字摘要包含更多细节，在信息覆盖度上优于 100 字摘要。
- (3) **ROUGE 的适用边界**：这一结果也说明 ROUGE 不应作为摘要质量的唯一指标，需结合人工评估综合判断。

5.2 人工评分

选取 5 篇文章，从流畅性（Fluency）、信息覆盖度（Coverage）、简洁性（Conciseness）三个维度进行 1-5 分人工评分。

表 4: 人工评分结果 (5 篇文章, 1-5 分)

维度	TextRank	LLM	LLM 优势
流畅性	5.0	4.6	-8%
信息覆盖度	4.0	4.8	+20%
简洁性	4.4	4.8	+9%
综合均分	4.47	4.73	+6%

结果分析:

- (1) **TextRank 流畅性满分**: 因为它直接摘抄原文句子, 句子本身天然通顺。但这恰恰暴露了抽取式摘要的本质——它不生产语言, 只搬运语言。
- (2) **LLM 信息覆盖度明显领先** (4.8 vs 4.0): TextRank 的一个代表性失败案例是, 在 36 氪 WAVES2026 峰会报道中, 它抽取了主持人对话片段而非核心观点; 而 LLM 成功归纳了“投早投小”“技术路线选择”“资本周期错配”等关键议题。
- (3) **人工评分差距远小于 ROUGE 差距**: 人工综合评分 LLM 仅领先 6%, 而 ROUGE 领先 170%。这说明 ROUGE 作为表面匹配指标, 可能低估了抽取式摘要的实际可读性。

5.3 分类准确率

从四分类中各选 5 篇, 共 20 篇文章, 人工验证 LLM 分类的准确性。

表 5: 分类准确率 (20 篇文章)

类别	样本数	正确数	准确率
AI/科技	5	5	100%
创业/商业	5	5	100%
编程/技术	5	5	100%
其他	5	5	100%
总计	20	20	100%

20 篇文章全部正确分类, 说明: (1) 预设的四分类标签与这批文章的领域分布高度契合; (2) Qwen2.5:7B 对科技类新闻的语义理解能力足以胜任该分类任务。

5.4 方案对比总结

表 6: TextRank vs LLM 方案全面对比

维度	TextRank	LLM 生成式
实现方式	自实现 80 行	Prompt 工程
推理速度	<1 秒/篇	3-8 秒/篇
ROUGE-L F1	0.1322	0.3571
人工评分	4.47	4.73
硬件依赖	无	GPU (8GB 显存)
能否离线	是	需加载模型
语言能力	仅抽取原文	理解 + 归纳 + 翻译

综合评估，LLM 生成式在所有质量指标上均优于 TextRank，代价是推理速度较慢和硬件依赖。实际部署中可考虑混合策略：对时效性要求高的场景使用 TextRank 快速出摘要；对质量要求高的场景使用 LLM 精加工。

6 总结与心得

6.1 项目收获

通过本项目，我们完整体验了 NLP 应用从数据获取、算法实现、前端展示到量化评估的全流程。

在实践中深刻理解了抽取式摘要与生成式摘要的本质差异——这不是简单的“好坏”之分，而是**忠实度**与**可读性**之间的权衡。TextRank 绝不编造，但缺乏概括；LLM 流畅智能，但可能产生幻觉。这一认知远比课堂上的理论讲解来得深刻。

Prompt Engineering 被证明是 NLP 工程实践中的核心技能——一个精心设计的 System Prompt 能让一次 API 调用同时完成分类和三种长度摘要，大幅减少 token 消耗和响应延迟。结构化 JSON 输出让 LLM 从“聊天工具”变成了“可编程组件”。

6.2 遇到的技术问题

- (1) **Python 3.13 兼容性**: newspaper3k 依赖的 feedparser 在 Python 3.13 下因 sgmlib 移除而无法导入。解决方案是将正文提取迁移到 trafilatura，RSS 解析改用 lxml.etree 直接处理 XML，完全消除了对 feedparser 的依赖。
- (2) **Ollama Python SDK 故障**: ollama 官方 Python 库在 Windows 环境下返回 HTTP 502 错误，但通过 curl 直接调用 API 正常。排查后改用 requests 库直接调用 Ollama 的 HTTP API，问题解决。

- (3) **数据库路径问题**: Streamlit 运行时的 CWD 与模块所在目录不一致, 导致 SQLite 数据库被创建到了错误的位置。解决方案是将 DB_PATH 改为基于 config.py 文件位置的绝对路径, 确保无论从哪里启动, 数据库都在固定位置。

6.3 未来改进方向

- (1) **扩展信息源**: 接入微信公众号、知乎、GitHub Trending 等渠道, 扩大覆盖面
- (2) **个性化推荐**: 基于用户阅读历史和反馈, 调整摘要详略和推送频率
- (3) **推送能力**: 增加邮件/微信推送, 实现真正的“订阅”闭环
- (4) **模型升级**: 使用更大参数量的模型 (如 Qwen2.5:14B) 或云端 API (GPT/Claude) 进一步提升摘要质量
- (5) **多模态摘要**: 支持视频/播客转写后的摘要

参考文献

- [1] Mihalcea R, Tarau P. *TextRank: Bringing Order into Text*. Proceedings of EMNLP, 2004.
- [2] Page L, Brin S, Motwani R, Winograd T. *The PageRank Citation Ranking: Bringing Order to the Web*. Stanford InfoLab, 1999.
- [3] Lin C Y. *ROUGE: A Package for Automatic Evaluation of Summaries*. Proceedings of ACL Workshop on Text Summarization, 2004.
- [4] Vaswani A, Shazeer N, Parmar N, et al. *Attention Is All You Need*. Proceedings of NeurIPS, 2017.
- [5] Qwen Team, Alibaba Cloud. *Qwen2.5 Technical Report*. 2025.
- [6] Barbaresi A. *Trafilatura: A Web Scraping Library and Command-Line Tool for Text Discovery and Extraction*. Proceedings of ACL: System Demonstrations, 2021.